Competitive Exam Practice 7: Operations on Processes

GATE (Computer Science) Pattern

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Concepts That Click

Each question carries 1–2 marks. MCQ = single correct option; NAT = Numerical Answer Type. In every program, assume all headers are included, every `fork` succeeds, and the program ends by reaping its children. *All questions in this set are original, written in the GATE pattern; none is claimed to be a past-year question.*

Q1 *[Original, GATE-pattern]* **[MCQ, 1 mark]** Immediately after a successful `fork()` call returns, which statement is **TRUE**?

(A) Both processes receive the child's pid

(B) The child receives `0` and the parent receives the child's pid

(C) The parent receives `0` and the child receives the parent's pid

(D) The child begins execution at the start of `main`

Q2 *[Original, GATE-pattern]* **[MCQ, 1 mark]** A process with pid 4100 calls `execlp("ls", "ls", NULL)` and the call succeeds. Which statement is **TRUE**?

(A) A new process is created to run `ls`, and pid 4100 continues after the call

(B) `ls` runs with pid 4100, and the statements after `execlp` in the original program are never executed

(C) `ls` runs with a new pid, and the original process becomes its parent

(D) `execlp` returns `0` after `ls` finishes

Q3 *[Original, GATE-pattern]* **[MCQ, 1 mark]** A *zombie* process is one that:

(A) is running but whose parent has terminated

(B) has terminated, and whose parent has not yet called `wait`

(C) is blocked forever waiting for I/O

(D) has been swapped out by the medium-term scheduler

Q4 *[Original, GATE-pattern]* **[MCQ, 2 marks]** On a UNIX system, a parent exits while its child is still running and the parent never called `wait`. Which statement is **TRUE**?

(A) The child is terminated immediately, on every UNIX system

(B) The child becomes a zombie at once and stops running

(C) The child is re-parented to `init` (pid 1), which later reaps it

(D) The child's open file descriptors are closed, but it continues as an unreapable process forever

Q5 *[Original, GATE-pattern]* **[NAT, 1 mark]** Consider the fragment

```
for (i = 0; i < 6; i++) fork();
```

The number of *child* processes created (not counting the original process) is _____.

Q6 *[Original, GATE-pattern]* **[NAT, 2 marks]** Consider the fragment

```
fork() && (fork() || fork()); printf("s\n");
```

The number of times `s` is printed is _____.

Q7 *[Original, GATE-pattern]* **[NAT, 2 marks]** Consider the fragment

```
for (i = 0; i < 4; i++) { fork(); printf("A\n"); fflush(stdout); }
```

The total number of lines printed is _____.

Q8 *[Original, GATE-pattern]* **[NAT, 2 marks]** Standard output is redirected to a file, and the program

```
printf("X"); fork(); fork(); fork(); printf("\n");
```

contains no call to `fflush`. The number of `X` characters in the file is _____.

Q9 *[Original, GATE-pattern]* **[NAT, 2 marks]** Consider the program

```
int x = 5; if (fork() == 0) x = x * 2; else { wait(NULL); x = x + 3; }
                            printf("%d\n", x);
```

The sum of all values printed is _____.

Q10 *[Original, GATE-pattern]* **[NAT, 2 marks]** A child process calls `exit(300)`. Its parent collects the status with `waitpid(pid, &st, 0)`. On a system where only the low-order 8 bits of the exit argument are delivered to the parent, the value of `WEXITSTATUS(st)` is _____.

Concepts That Click